DESARROLLO DE APLICACIONES WEB · EXAMEN FINAL

Laboratorio de seguridad

Diagnóstico, controles aplicados y respuestas finales

All In La PK · API Express y tienda React

EQUIPO	Nicolay Baquero Santiago Cifuentes
FECHA	28 de octubre de 2026
REPOSITORIO	SCC405/all-in-la-pk · rama seguridad

El informe distingue lo ya comprobado de lo que requiere acceso a Atlas o un despliegue posterior. No se incluyen secretos ni certificados privados.

1. Diagnóstico inicial

Pregunta de la guía	Resultado al iniciar	Evidencia y alcance
¿Contraseñas guardadas en claro?	No	La única cuenta es la del administrador; ADMIN_PASSWORD_HASH usa scrypt con sal (backend/src/utils/password.js). No hay colección de usuarios ni registro público.
¿Credenciales reales escritas en código?	No encontradas	La configuración usa variables de entorno. La búsqueda en el árbol y el historial encontró ejemplos de URI mongodb+srv://usuario:clave@cluster.mongodb.net en pruebas, no credenciales reales. Esto no equivale a una auditoría forense completa de secretos.
¿Archivos de secretos versionados?	No	git ls-files solo encuentra .env.example; backend/.env y backend/certs/ están ignorados.
¿Texto de usuarios insertado como HTML?	No	En frontend/src no se usa dangerouslySetInnerHTML, innerHTML, outerHTML ni document.write para mostrar el catálogo; React interpola los valores como texto.
¿IDOR entre clientes cambiando un identificador?	No aplica	No hay cuentas de clientes, pedidos persistidos ni GET /api/pedidos/:id. Los identificadores de productos y categorías son públicos para lectura.
¿Administración protegida solo al ocultar botones?	No	El middleware crearRequiereSesion exige sesión firmada en POST, PUT y DELETE de categorías y productos. Las consultas permanecen públicas.
¿CORS abierto a cualquier origen?	No	backend/src/app.js usa env.corsOrigin, no *. En Render el origen configurado es la URL exacta de la tienda.
¿Se usa HTTP simple?	Sí, opcional en desarrollo; no en producción	El servidor local puede arrancar por HTTP para desarrollo. Ya existía HTTPS local con certificado y redirección 308 al activar FORZAR_HTTPS. Render sirve la API y la tienda por HTTPS y anuncia HSTS.

Modelo real del proyecto. El login es para un único administrador. No existe registro de clientes, ni un rol cliente, ni órdenes guardadas en MongoDB. La pantalla de compra reúne datos temporalmente en el navegador y construye una confirmación sin enviar un pedido a la API. Por ello, los ejemplos de registro, cambio de rol e IDOR de la guía se califican como **no aplicables**, no como pruebas aprobadas.

2. Hallazgos y correcciones

#: 1

Hallazgo: El login no limitaba los intentos.

OWASP 2025: A07, A09

Comprobación: Antes, POST /api/sesion no tenía limitador.

Corrección o tratamiento: express-rate-limit: cinco fallos por IP cada 15 minutos; el sexto responde 429. Se registran los fallos y el bloqueo sin escribir credenciales.

Estado: Corregido en la rama.

#: 2

Hallazgo: El token de sesión duraba ocho horas.

OWASP 2025: A07

Comprobación: DURACION_MS en backend/src/middlewares/sesion.js.

Corrección o tratamiento: Reducido a 15 minutos; la caducidad está firmada y la cookie tiene el mismo límite.

Estado: Corregido en la rama.

#: 3

Hallazgo: Faltaban cabeceras defensivas de Helmet y una CSP explícita para la tienda.

OWASP 2025: A02, A05

Comprobación: La API solo quitaba X-Powered-By; el sitio estático no declaraba CSP en render.yaml.

Corrección o tratamiento: Helmet en la API, CSP de Swagger ajustada a su script en línea y CSP estricta de la tienda en el Blueprint de Render.

Estado: Preparado en la rama; la CSP del sitio requiere fusionar y aplicar el Blueprint en Render.

#: 4

Hallazgo: Dependencias con avisos de seguridad en la instalación inicial.

OWASP 2025: A03

Comprobación: npm audit detectó avisos en el árbol de body-parser, qs y vite.

Corrección o tratamiento: npm audit fix en backend y frontend, con archivos lock actualizados. Auditorías posteriores: cero vulnerabilidades en ambos.

Estado: Corregido en la rama; repetir auditoría periódicamente.

#: 5

Hallazgo: Producción podía arrancar con secretos de firma efímeros si faltaba una variable.

OWASP 2025: A02, A04

Comprobación: csrf.js y sesion.js generaban un secreto por proceso como respaldo.

Corrección o tratamiento: El arranque de producción ahora exige MONGODB_URI, CORS_ORIGIN, CSRF_SECRET, ADMIN_USUARIO, ADMIN_PASSWORD_HASH y SESION_SECRET; solo informa los nombres ausentes. Desarrollo conserva el respaldo local.

Estado: Corregido en la rama.

#: 6

Hallazgo: Un precio negativo manipulado en localStorage superaba la validación de forma del carrito.

OWASP 2025: A06, A08

Comprobación: itemValido comprobaba que el precio fuera finito, pero no que fuera no negativo.

Corrección o tratamiento: Se rechazan precios negativos al restaurar el carrito; ya se rechazaban cantidades menores que uno. Hay prueba para ambos valores.

Estado: Corregido en la rama.

#: 7

Hallazgo: La configuración de acceso de Atlas no se pudo verificar en esta sesión.

OWASP 2025: A02, A04

Comprobación: El panel de Atlas solicitó autenticación al intentar revisar Network Access y Database Access.

Corrección o tratamiento: Verificar en la cuenta del proyecto el rango de IP y que el usuario de la API tenga solo readWrite sobre all_in_la_pk; no alterar la lista de IP sin validar el egreso de Render.

Estado: Pendiente de comprobación del titular.

#: 8

Hallazgo: El checkout solicita datos personales aunque el pedido no se transmite ni persiste.

OWASP 2025: A06, privacidad

Comprobación: frontend/src/pages/Compra.jsx pide nombre, correo, teléfono, dirección y ciudad; construirPedido los conserva en el estado de navegación para la confirmación.

Corrección o tratamiento: No guardar datos de tarjeta; antes de un uso real, explicar el carácter de demostración o implementar un proceso de pedido con finalidad, autorización y conservación definidas.

Estado: Riesgo residual de diseño.

HTTPS y cookies

Se generó un certificado autofirmado local con el script existente npm run certificados --prefix backend; la clave y el certificado quedaron bajo backend/certs/, ignorados por Git. La prueba de HTTPS local pasó. El servidor ya soporta TLS directo y redirección HTTP→HTTPS al activar la configuración; la comprobación de salud mantiene su excepción para Render y HSTS sigue activo sobre la conexión segura. Una petición de solo lectura a la API desplegada devolvió 200 y Strict-Transport-Security: max-age=31536000; includeSubDomains. El sitio estático desplegado también respondió por HTTPS. El certificado local es autofirmado: no se debe omitir la advertencia del navegador ni confundirlo con un certificado público.

La cookie de sesión es HttpOnly y Secure en producción. Aquí SameSite=Strict no funciona porque la tienda y la API tienen dominios onrender.com diferentes; por eso se usa SameSite=None; Secure, con token CSRF firmado, doble envío y control del encabezado Origin. En desarrollo se usa Lax. Esta excepción respecto al ejemplo de la guía es necesaria para que funcione el despliegue real y queda compensada por la defensa CSRF.

Pregunta de HTTPS. HTTPS da confidencialidad del tráfico e integridad/autenticación del servidor mediante el certificado (sujetas a que la cadena de confianza se valide). HTTP no ofrece esas garantías. HTTPS no demuestra que el contenido, la empresa o el vendedor sean honestos.

Autenticación, roles e IDOR

El hash scryptsalderivada se guarda en la variable de entorno ADMIN_PASSWORD_HASH, no en una tabla de usuarios. El token de sesión tiene firma HMAC y caduca a los 15 minutos. El mensaje de credenciales es genérico. La prueba automatizada confirma 401 en cinco fallos y 429 en el sexto, sin emitir cookie de sesión. Una solicitud de modificación sin sesión pero con CSRF válido responde 401; sin CSRF responde 403 porque esa capa se evalúa primero.

El único privilegio es el de administrador configurado por el servidor. No hay endpoint de registro donde enviar rol: "admin", ni pedidos de cliente para una prueba IDOR entre A y B. Añadir endpoints ficticios solo para hacer coincidir la guía ampliaría la superficie de ataque sin resolver un requisito del proyecto.

XSS, CSRF y entradas manipuladas

Los nombres y descripciones de productos se renderizan como texto de React. Una cadena de prueba como <img src=x onerror="alert('XSS')"> se mostraría literalmente; no se encontró un punto de renderizado HTML directo. La CSP propuesta para el sitio solo permite scripts del propio origen y bloquea manejadores en línea. La CSP de Swagger conserva unsafe-inline para el script que genera Swagger UI, una excepción acotada a /api-docs.

Las operaciones que modifican datos exigen la cabecera X-CSRF-Token, una cookie firmada coincidente y un origen permitido. Los tests existentes cubren el rechazo 403 sin token o con token inválido. **Pregunta de CSRF:** incluso con SameSite=Strict, el token agrega defensa en profundidad ante diferencias entre navegadores y escenarios de subdominios. En este despliegue se necesita además porque la cookie es SameSite=None.

La guía propone enviar una compra con cantidad -5 o precio manipulado. Este proyecto no tiene endpoint de compra: no hay servidor que acepte un total fraudulento. Se probó la superficie equivalente disponible, el carrito restaurado desde localStorage: ahora rechaza cantidad: -5 y precio: -100, y sincroniza los precios aceptados con

el catálogo al cargarlo. Si se implementa un pedido real, **el servidor deberá volver a leer precios y stock de MongoDB y calcular allí el total**; la validación del navegador no autoriza un cobro.

3. Revisión OWASP Top 10:2025

Categoría	Resultado en All In La PK
A01 Control de acceso roto	Mutaciones de catálogo protegidas en Express; no existe recurso privado por propietario para probar IDOR.
A02 Configuración insegura	CORS específico y HSTS existentes; Helmet/CSP añadidos. Atlas y aplicación efectiva de la CSP en Render pendientes de verificar.
A03 Fallos de cadena de suministro	npm audit de ambos proyectos: cero avisos tras actualizar dependencias.
A04 Fallos criptográficos	HTTPS y hash lento con sal; secretos fuera del código. Arranque de producción falla si falta uno.
A05 Inyección	React escapa texto y los esquemas de Mongoose validan tipos; CSP de la tienda preparada.
A06 Diseño inseguro	Restauración del carrito endurecida; sigue faltando un flujo real de pedidos y tratamiento definido de datos del comprador.
A07 Fallos de autenticación	Login genérico, token de 15 minutos y límite de cinco fallos por IP.
A08 Integridad de software o datos	Firma HMAC de sesión y CSRF; la UI no debe ser fuente de verdad para un pedido futuro.
A09 Fallos de registro y alertas	Se registran fallos y bloqueos de login; no hay alertas ni centralización de seguridad.
A10 Manejo de excepciones	Respuestas 500 genéricas y errores de conexión sin filtrar URI; faltantes de producción impiden iniciar.

4. Riesgos residuales

Riesgo que sigue presente: El usuario de Atlas o su lista de IP podrían tener permisos amplios; no se pudo entrar al panel.
Probabilidad: Media
Impacto: Alto
Tratamiento: Mitigar: revisar Network Access y limitar la cuenta de la API a readWrite en la base del proyecto.

Riesgo que sigue presente: La CSP del frontend solo entrará en vigor cuando Render aplique el Blueprint actualizado.
Probabilidad: Media
Impacto: Medio
Tratamiento: Mitigar: sincronizar el servicio y verificar la cabecera y la consola del navegador después del despliegue.

Riesgo que sigue presente: El login usa un limitador en memoria por IP; reinicios o varias instancias reinician el conteo y un origen compartido puede bloquear a otros.

Probabilidad: Media

Impacto: Medio

Tratamiento: Mitigar: usar almacén compartido y alertas si escala el despliegue.

Riesgo que sigue presente: El token de sesión es autocontenido; cerrar sesión borra la cookie, pero no revoca un token robado antes de que venza.

Probabilidad: Baja

Impacto: Alto

Tratamiento: Mitigar: lista de revocación o sesiones del servidor, además del vencimiento de 15 minutos.

Riesgo que sigue presente: El checkout captura datos personales de forma transitoria sin un proceso de pedido real.

Probabilidad: Media

Impacto: Medio

Tratamiento: Evitar en una versión pública de uso real hasta definir finalidad, aviso y persistencia segura; reducir campos en la demostración.

5. Plan ante filtración de la URI de Atlas

Se sigue el ciclo de preparación, detección, contención, erradicación, recuperación y lecciones aprendidas solicitado en la guía, consistente con la respuesta a incidentes de NIST. Una URI expuesta debe tratarse como comprometida aunque todavía no se vea abuso.

1. **Preparación:** mantener responsables (Nicolay y Santiago), acceso al panel de Atlas y Render, copia de seguridad, registro de variables y procedimiento de rotación sin secretos en Git ni por chat.

2. **Detección y análisis:** identificar dónde apareció la URI (commit, log o mensaje), desde cuándo es visible, qué permisos tenía su usuario y si los registros de Atlas muestran conexiones o cambios anómalos. Conservar evidencias sin volver a difundir la clave.

3. **Contención:** deshabilitar o rotar inmediatamente el usuario/contraseña de Atlas filtrado, restringir acceso de red según el egreso real del servicio y retirar la URI de superficies públicas. Coordinar una pausa breve si hace falta para evitar nuevas escrituras.

4. **Erradicación:** sustituir la credencial en Render, eliminar copias del código y del historial con el procedimiento adecuado; revisar otras credenciales que compartieran el mismo secreto. Borrar una línea de un commit **no** reemplaza la rotación.

5. **Recuperación:** desplegar con la nueva URI, verificar conexión, integridad de categorías y productos, privilegios mínimos y acceso normal; observar logs y comparar con la copia de seguridad.

6. **Lecciones aprendidas:** registrar línea de tiempo y alcance, activar análisis automático de secretos en PR, revisar permisos y políticas de rotación; evaluar obligaciones de notificación si se confirmara exposición de datos personales.

6. Datos personales y Ley 1581 de 2012

El catálogo y las categorías se guardan en Atlas; **no hay colección de clientes ni de pedidos**. La pantalla de compra solicita nombre, correo, teléfono, dirección y ciudad, y los mantiene temporalmente en el estado del navegador para mostrar la confirmación. El carrito guardado en localStorage contiene productos, cantidades y

precios, no esos datos personales. Tampoco se solicitan datos de tarjeta. Como no hay proceso real de envío o pago, dirección y teléfono sobran para una demostración; incluso correo y nombre deben pedirse solo si existe una finalidad real comunicada al usuario. Antes de convertirlo en una tienda operativa hacen falta aviso de finalidad, autorización cuando corresponda, conservación y control de acceso acordes con los principios de finalidad, libertad, circulación restringida y seguridad del artículo 4 de la Ley 1581 de 2012. Esto es un análisis de alcance del prototipo, no una declaración de cumplimiento legal.

7. Respuestas finales

1. **Hallazgo más grave:** el login sin límite de intentos permitía probar contraseñas repetidamente contra la única cuenta capaz de modificar el catálogo. Se corrigió con cinco fallos por 15 minutos, 429 al sexto y un token de vida corta; el limitador en memoria sigue siendo una medida parcial.

2. **Defensa frente al caso Uber de la clase:** una URI o clave expuesta en un repositorio requiere detección de secretos antes del commit, privilegios mínimos y rotación inmediata al descubrir la filtración. Quitar el texto del último commit sin cambiar la credencial no elimina el acceso que ya pudo copiarse.

3. **Mejora futura:** añadir MFA al administrador, sesiones revocables y registros/alertas centralizados. Antes de procesar pedidos reales, validar precios y cantidades en el servidor y definir el tratamiento de datos personales.

8. Evidencia de verificación y límites

• npm test --prefix backend: **110 aprobadas, 3 omitidas, 0 fallidas**. Las omitidas requieren MONGODB_URI_TEST sobre una base terminada en _test; no se conectó la suite destructiva al Atlas compartido.

• npm test --prefix frontend: **136 aprobadas, 0 fallidas**. npm run build --prefix frontend compiló con Vite 8.3.5.

• Swagger UI se abrió en un servidor local con la nueva CSP: se renderizaron el título, los esquemas y las rutas de categorías, productos y sistema.

• npm audit --prefix backend y npm audit --prefix frontend: **0 vulnerabilidades** en cada uno tras actualizar los lockfiles.

• npm outdated muestra actualizaciones disponibles, entre ellas Mongoose 9 y React 19; no se hizo una migración mayor dentro de este laboratorio porque requiere pruebas de compatibilidad. Una versión más nueva no implica por sí sola una vulnerabilidad en la versión instalada.

• git ls-files y git status --ignored: .env y certificados locales permanecen ignorados. La búsqueda de URI en el historial solo encontró las cadenas de ejemplo de backend/test/ayuda-base-de-datos*.

• Lecturas HTTPS del 28-10-2026: /api/health respondió 200 con HSTS; el frontend respondió 200. Estas lecturas comprueban la versión **actualmente desplegada**, no los cambios de esta rama.

• Pendientes de comprobación externa: permisos y rango de IP de Atlas (el panel pidió iniciar sesión); aplicación de la nueva CSP y de las dependencias actualizadas en Render después de fusionar y desplegar. No se registran como verificados.

Fuentes de referencia: guía del laboratorio entregada por el docente; OWASP Top 10:2025; NIST, respuesta a incidentes; Ley 1581 de 2012, artículo 4.